

Studienarbeit
**Implementierung eines digitalen
Signalprozessors für den Audiobereich**

im Studiengang Technische Informatik
der Fakultät Informationstechnik
Sommersemester 2025

Nuro Savelsberg

Zeitraum: 01.03.2024 - 14.04.2025
Prüfer: Prof. Dr.-Ing. Clemens Klöck
Zweitprüfer: Prof. Dr.-Ing. Thao Dang

Inhaltsverzeichnis

1	Überblick	1
2	Motivation	1
2.1	Ausgangssituation	1
2.1.1	Mängel	3
2.2	Ziele	4
3	Grundlagen	4
3.1	I2S	5
3.2	Digitale Filter	5
4	Realisierung	6
4.1	Ansatz	6
4.2	Hardware Aufbau	7
4.3	Programmstruktur	8
4.4	Implementierung digitaler Filter	8
4.5	Lautstärkesteuerung	11
4.6	Validierung	11
5	Ergebnisse	12
6	Zusammenfassung und Ausblick	13
	Literaturverzeichnis	15

1 Überblick

Dieser Bericht beschreibt die Entwicklung und Implementierung eines Digitale Signal Prozessor (DSP) für eine tragbare, akkubetriebene Musikbox. Das bestehende analoge Signalverarbeitungssystem der Musikbox wies einige Mängel auf, die durch die Weiterentwicklung behoben werden sollten. Das bisherige Signalverarbeitungssystem besteht aus analogen Butterworth-Bandpassfiltern. Die Hauptprobleme bestanden dabei aus der hohen Komplexität des Systems und aus der Dämpfung des Signalpegels im Durchlassbereich durch die analogen Filter. Dadurch war die maximale Lautstärke der Musikbox begrenzt. Die Verwendung eines DSP soll diese Probleme lösen und wird im Rahmen eines Studienprojekts als Teil des Studiums der Technischen Informatik durchgeführt.

Der Bau der Musikbox begann im Frühling 2022 und nach etwa vier Monaten wurde die erste Version fertiggestellt. Innerhalb der nächsten eineinhalb Jahre wurden etliche Weiterentwicklungen durchgeführt, um Probleme am Gehäuse und der Elektronik zu lösen und um die Klangqualität zu verbessern. Während die Musikbox immer ausgereifter wurde, konnten die Mängel am Signalverarbeitungssystem bis zu Beginn des hier beschriebenen Projekts nicht behoben werden.

Für die Umsetzung wurde ein Mikrocontroller (ESP32-S3-DevKitC-1) als DSP konfiguriert. Der Mikrocontroller empfängt das digitale Audiosignal über eine I2S-Schnittstelle von einem Bluetooth-Empfänger und verarbeitet es weiter. Nachdem das Signal gefiltert wurde und die Lautstärke angepasst wurde, werden die Signale an externe Digital Analog Converter (DAC) gesendet.

2 Motivation

2.1 Ausgangssituation

Der DSP soll in einem bestehenden Audiosystem eingesetzt werden. Dabei handelt es sich um eine tragbare, akkubetriebene Musikbox, die ein analoges Signalverarbeitungssystem verwendet (Abb.2.1). Die Musikbox verfügt über zwei Hochtöner, zwei Mitteltöner und einen Subwoofer. Die Hoch- und Mitteltöner verfügen über einen rechten und einen linken Audiokanal, insgesamt

werden also fünf Lautsprecher verwendet, die jeweils ein eigenes Signal erhalten, welches sich von dem der anderen Lautsprecher unterscheidet. An der Oberseite des Gehäuses befinden sich Drehknöpfe, um sowohl die Gesamtlautstärke als auch die Lautstärke der Hoch-, Mitteltöner und des Subwoofers unabhängig voneinander einstellen zu können. Auf derselben Platte sind auch der Hauptschalter, um das System an und auszuschalten und ein LCD-Display, welches die Spannung des Akkupacks und den fließenden Strom anzeigt.



Abb. 2.1: Die Musikbox in dem der DSP eingesetzt wird

Das Gehäuse ist vorwiegend aus Holz konstruiert und ist in drei voneinander abgegrenzte Kammern aufgeteilt. In der Hauptkammer sind der Akku, die Audioverstärker und alle sonstigen elektronischen Komponenten untergebracht. Diese Kammer ist durch eine abnehmbare Klappe an der Rückseite zugänglich. Des Weiteren dient die Hauptkammer als Resonanzraum für den Subwoofer. Die anderen beiden Kammern dienen jeweils als Resonanzraum für den rechten und den linken Mitteltöner. Während die Mitteltöner einen geschlossenen Resonanzraum verwenden, wird für den Subwoofer ein Bassreflex-Gehäuse eingesetzt. Ein Resonanzraum erzeugt bei einer bestimmten Frequenz eine Resonanz mit dem verwendeten Lautsprecher. Bei einem geschlossenen Resonanzraum hängt die Resonanzfrequenz von den Eigenschaften des Lautsprechers und vom Volumen des Gehäuses ab, während bei einem Bassreflex-Gehäuse zusätzlich auch die Fläche, Länge und Form des Ports entscheidend sind. Um die Eigenschaften der Gehäuse zu bestimmen, wurden Simulationen der Lautsprecher mit dem gewünschten Gehäusertyp durchgeführt. Für diese Simulationen wurde die Software [WinISD](#) verwendet. Die Hochtöner sind im Resonanzraum des Subwoofers montiert, da diese keinen Resonanzraum benötigen.

Das System wird durch einen Akkupack mit Strom versorgt. Dieses besteht aus 18 Zellen des Typs 18650. Die Zellen sind in Dreiergruppen unterteilt, die miteinander parallel verbunden sind. Sechs dieser Gruppen sind in Reihe verbunden, um die Spannung des Akkupacks auf 24 V zu bringen. Die Zellen werden durch ein Batterie Management System (BMS) geschützt, dieses schützt vor Überentladung und Überladung, limitiert den maximalen Entladestrom und sorgt

dafür, dass die Zellen gleichmäßig entladen und geladen werden. Geladen wird der Akkupack mit einem im Gehäuse montierten Netzteil.

Die Signale der Hoch- und Mitteltöner werden von einem Klasse-AB-Audioverstärker mit vier Kanälen verstärkt, während für den Subwoofer ein eigener Klasse-D-Verstärker eingesetzt wird. Das analoge Signalverarbeitungssystem nimmt das Eingangssignal, welches den gesamten Audiofrequenzbereich umfasst, entgegen und teilt es in drei Frequenzbänder für die jeweiligen Lautsprecher auf. Dafür werden analoge Butterworth-Bandpassfilter der zweiten Ordnung eingesetzt. Die Grenzfrequenzen der Filter sind so gewählt, dass jeder Lautsprecher nur genau den Frequenzbereich erhält, den dieser am besten wiedergeben kann. Würde jeder Lautsprecher den gesamten Frequenzbereich abspielen, würde das zu minderwertiger Klangqualität oder sogar zu Beschädigungen an den Lautsprechern führen. Zur Bestimmung der Grenzfrequenzen wurden Simulationen mit der Software [VituixCAD](#) durchgeführt.

2.1.1 Mängel

Ein Mangel des analogen Signalverarbeitungssystems ist die hohe Komplexität des Systems. Je länger der Signalpfad vom Bluetooth-Empfänger zu den Audioverstärkern ist, desto größer ist die Anzahl der möglichen Fehlerursachen. Des Weiteren steigt auch der Störeinfluss durch elektromagnetische Interferenz. Bei der bisherigen Implementierung läuft der Signalpfad vom Empfänger erst zu dem Potentiometer, um die Gesamtlautstärke zu steuern, anschließend durch die analogen Filter, danach durch die Potentiometer der einzelnen Frequenzbereiche und erst danach zu den Audioverstärkern. Da sich die Potentiometer an der Gehäuseoberseite befinden, während die restlichen elektrischen Komponenten an der Gehäuserückseite angebracht sind, müssen einige Kabel verwendet werden, um die Signale zu transportieren. Das erhöht die Angriffsfläche durch elektromagnetische Interferenz, und das Fehlerpotential wird durch die Anzahl der benötigten Lötstellen und sonstigen Verbindungen stark erhöht. Des Weiteren führt jedes qualitativ minderwertige Bauteil auf dem Signalpfad zur Verringerung der Signalqualität.

Zusätzlich zu einer verschlechterten Signalqualität erschwerte dieser Mangel den Entwicklungsprozess hochgradig und sorgte für zahlreiche aufwendige Fehlersuchen, die die Fertigstellung des Projekts verzögerten.

Ein weiteres Problem geht aus einer grundlegenden Eigenschaft analoger Filter hervor: Der Signalpegel der Signale wird auch im Durchlassbereich gedämpft. In diesem System hat das dazu geführt, dass die maximale Leistung der Audioverstärker und der Lautsprecher nicht ausgelastet werden konnte. Dadurch war die maximale Lautstärke maßgeblich limitiert.

Um diesem Mangel entgegenzuwirken, wurde nach der Fertigstellung des Systems eine Weiterentwicklung durchgeführt, um den Signalpegel zu verstärken. Dabei wurden Operationsverstärker verwendet, die in einer nicht-invertierenden Verstärkerschaltung eingesetzt wurden. Die Verstärkung konnte dabei über Potenziometer eingestellt werden. Die Verstärkung der verschiedenen Frequenzbereiche muss unabhängig voneinander festgelegt werden können, damit die

jeweilige Verstärkung so eingestellt werden kann, dass die maximale Leistung aller Lautsprecher ausgelastet, aber nicht überschritten wird. Dafür werden also fünf identische Schaltungen benötigt.

Dieser Lösungsansatz verstärkt das Problem der Komplexität noch weiter und sorgt für mehr mögliche Problemstellen. Da kein anderer Ansatz identifiziert werden konnte, der nicht mit deutlich mehr Aufwand verbunden war, wurde dieser Ansatz trotzdem gewählt. Nach dem Einbau der Operationsverstärkerschaltung zeigten sich einige verheerende Probleme. Auf einigen Audiokanälen war ein starkes Rauschen zu hören und andere waren komplett ohne Ton, während nur wenige Kanäle einwandfrei funktionierten. Auf den funktionierten und den verrauschten Kanälen war der gewünschte Verstärkungseffekt klar zu erkennen, die Schaltung war so aber natürlich trotzdem nicht einsetzbar.

Als mögliche Ursachen für diese Probleme wurden Mängel bei der Herstellung wie schlechte Lötstellen oder ähnliche Probleme vermutet. Des Weiteren verkomplizierte die große Anzahl der Fehlerursachen die Identifikation der Fehler hochgradig. Nachdem auch nach einer ausgiebigen Fehlersuche die Mängel nicht identifiziert und behoben werden konnten, wurde der Ansatz ein analoges Signalverarbeitungssystem zu verwenden, gänzlich verworfen.

2.2 Ziele

Das Hauptziel, das sich durch die Entwicklung eines digitalen Signalverarbeitungssystems erhofft wird, ist die Behebung der Probleme des analogen Systems. Der Signalpfad des analogen Signals soll auf ein Minimum reduziert werden und die Verstärkung der einzelnen Frequenzbereiche soll unabhängig voneinander gesteuert werden können.

Dabei soll jede Funktionalität des bisherigen Systems beibehalten werden. Dazu zählen die Drehknöpfe für die Hauptlautstärke sowie für die Lautstärke aller Frequenzbereiche. Die Potentiometer dieser Drehknöpfe sollen vom DSP eingelesen und auf das digitale Signal angewandt werden, um den analogen Signalpfad so simpel wie möglich zu halten.

Der Bluetooth-Empfänger, welcher ein analoges Signal ausgibt, soll durch einen Empfänger ersetzt werden, der digital mit dem DSP kommuniziert. Dadurch muss das Signal nicht mehrmals analog und wieder digital gewandelt werden.

3 Grundlagen

3.1 I2S

Als Kommunikationsprotokoll für die Implementierung wurde der Inter-IC Sound Bus (I2S) gewählt. I2S wurde 1986 von Philips Semiconductor (heute NXP Semiconductors) entworfen. Dabei handelt es sich um eine serielle Schnittstelle zur Übertragung von digitalen Audiosignalen [3].

Der Bus verwendet drei Leitungen: Ein Taktsignal (Continuous Serial Clock (SCK)), ein Datensignal (Serial Data (SD)) und ein Signal, um zwischen dem rechten und linken Audiokanal zu wechseln (Word Select (WS)). Alle Signale können sowohl vom Bus-Master als auch vom Bus-Slave generiert werden, aber vor allem die Taktsignale (SCK und WS) werden meist vom Bus-Master geschrieben. Da der Bus keine Adressierung verwendet, müssen Implementierungen, die mehrere sendende Busteilnehmer beinhalten, mehrere separate Datenleitungen einsetzen. In [Abb.3.1](#) ist ein Zeitverlaufdiagramm des Protokolls abgebildet.

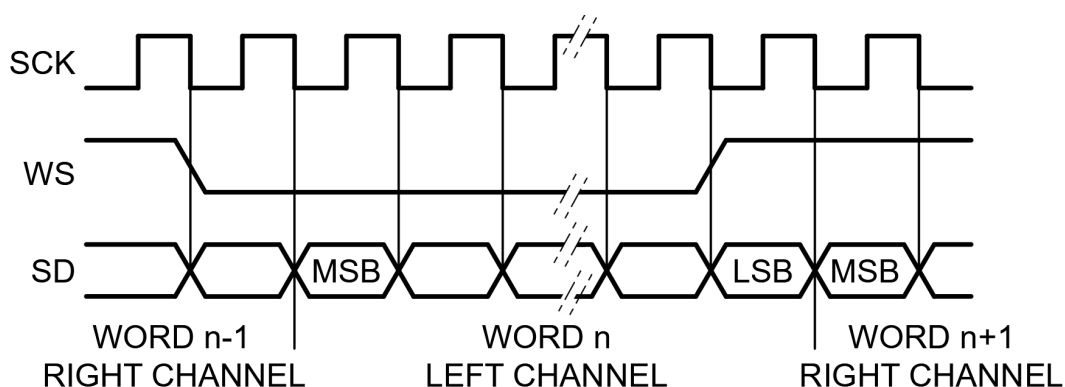


Abb. 3.1: I2S Zeitverlaufdiagramm [3]

3.2 Digitale Filter

Der DSP verwendet digitale Butterworth-Filter, um das Eingangssignal in einzelne Teilsignale aufzuteilen. Butterworth-Filter weisen eine besonders flache Frequenzantwort auf [1]. Das bedeutet, dass es weder im Durchlassbereich noch im Sperrbereich Welligkeiten gibt. Gleichzeitig lassen sie sich simpel sowohl analog als auch digital implementieren. Das macht sie zu einer beliebten Wahl für verschiedene Anwendungsbereiche, inklusive in der Audiotechnik.

Die allgemeine Übertragungsfunktion $H(s)$ eines Tiefpassfilters zweiter Ordnung lautet:

$$H(s) = \frac{Y(s)}{X(s)} = \frac{\omega_0^2}{s^2 + \sqrt{2}\omega_0 s + \omega_0^2} \quad (3.1)$$

Dabei ist s die komplexe Kreisfrequenz und ω_0 die Grenzfrequenz.

Um die zeit-diskrete Übertragungsfunktion $H(z)$ zu erhalten, wird die bilineare Transformation verwendet. Dafür wird $s = 2f_s \frac{z-1}{z+1}$ gesetzt [2]. Dabei ist f_s die Frequenz, bei der der Filter berechnet wird. Die resultierende Übertragungsfunktion hat die Form:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \dots}{1 - a_1 z^{-1} - a_2 z^{-2} + \dots} \quad (3.2)$$

Auf Mikrocontrollern werden digitale Filter meist im Zeitbereich angewandt. Dadurch entfällt die Notwendigkeit für eine Transformation in den Frequenzbereich, was Rechenleistung spart. Mithilfe der Koeffizienten a_n und b_n kann das Ergebnis des Filters y_n anhand vergangener Ergebnisse und Eingangssignale x_n berechnet werden:

$$y_n = a_1 y_{n-1} + a_2 y_{n-2} + \dots + b_0 x_n + b_1 x_{n-1} + \dots \quad (3.3)$$

4 Realisierung

4.1 Ansatz

Es gibt einige kostengünstige programmierbare DSPs für den Audibereich, jedoch konnte keiner identifiziert werden, der sowohl über eine Schnittstelle für ein digitales Eingangssignal verfügt als auch genug Flexibilität in der Programmierung zulässt, um die Drehknöpfe zu implementieren.

Bei dem gewählten Lösungsansatz wird ein Mikrocontroller verwendet, der als DSP konfiguriert ist. Viele Mikrocontroller verfügen über Peripheralschnittstellen für digitale Kommunikation mit verschiedenen Protokollen sowie über eine große Auswahl an GPIO Pins, über die Potentiometer eingelesen werden können. Um das digitale Signal in ein Analoges zu wandeln sollen externe DAC eingesetzt werden.

Einige Mikrocontroller sind mit Bluetooth-Antennen ausgestattet, die verwendet werden könnten, um das Eingangssignal zu empfangen. Um die Komplexität des Projekts zu limitieren wurde

sich jedoch dazu entschieden, einen externen Empfänger zu verwenden. Dadurch ist die Kompatibilität mit jeglichen Bluetooth-Sendegeräten gewährleistet, ohne dass diese speziell entwickelte Software verwenden müssen.

4.2 Hardware Aufbau

Als Empfänger wurde der TinySine Bluetooth AudioB I2S Empfänger gewählt. Dieser Empfänger gibt das empfangene Signal über eine I2S Schnittstelle mit bis zu 48 kHz und einer Auflösung von 32-Bit aus. Der Empfänger erlaubt zudem den Einsatz von Tastern, um das aktuelle Lied zu pausieren, weiterlaufen zu lassen, überspringen oder zu wiederholen.

Als DAC wurde die GY-PCM5102 DAC Platine verwendet. Diese verwendet den PCM5102A DAC Chip von Texas Instruments. Der DAC unterstützt bis zu 384 kHz bei 32-Bit und verfügt ebenfalls über eine I2S-Schnittstelle. Da diese Platine über zwei Kanäle verfügt, müssen hier mehrere eingesetzt werden.

Bei der Wahl des Mikrocontrollers wurde sich für den ESP32-S3-DevKitC-1 von Espressif Systems entschieden. Dieser zeichnet sich vor allem durch einen *dual-core* Prozessor aus, der mit bis zu 240 MHz taktet. Des Weiteren verfügt er über 8MB Flash Speicher, 2MB PSRAM, zwei I2S Peripherieschnittstellen und eine Vielzahl an GPIO Pins. Da dieser Mikrocontroller nur über zwei I2S Peripherieschnittstelle verfügt, können nur vier Kanäle angesteuert werden. Um trotzdem den ESP32-S3 verwenden zu können, musste ein Kompromiss eingegangen werden. Dafür wurden die Hochtöner nicht mehr in *stereo*, sondern in einer *mono* Konfiguration eingesetzt. Da sich die Hochtöner nun denselben Kanal teilen können, sind nur noch vier Kanäle benötigt. Dadurch, dass die Hochtöner mit einem Abstand von weniger als 300 Millimetern zueinander montiert sind, war der *stereo* Effekt auch zuvor nur bei geringer Distanz zur Musikbox zu erkennen.

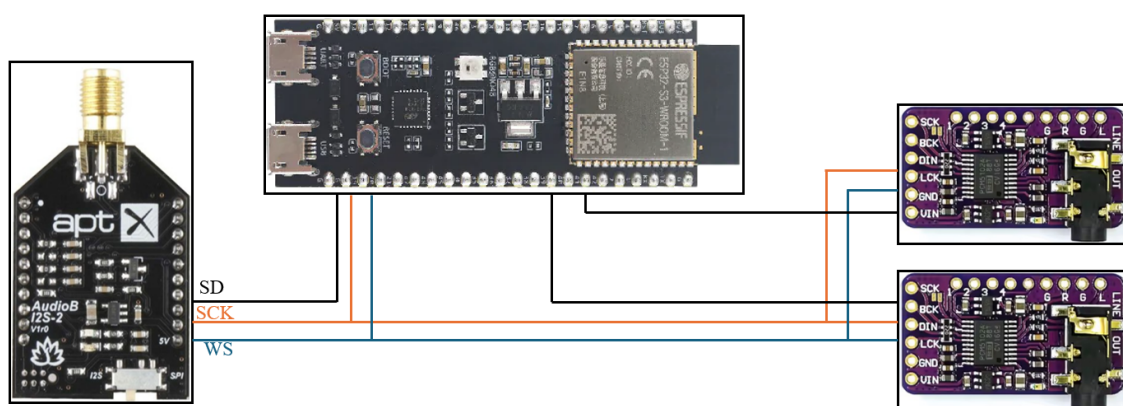


Abb. 4.1: Hardware Aufbau der relevanten Komponenten. Links der Bluetooth-Empfänger, mittig der Esp32s3 und rechts die zwei DACs

Diese Komponenten sind alle über einen I2S-Bus verbunden. Da I2S keine Adressierung unterstützt, wird bei den Datenleitungen eine Punkt-zu-Punkt-Verbindung zwischen den jeweiligen Komponenten verwendet. Im Gegensatz dazu sind die Komponenten über dieselben zwei Taktleitungen verbunden. Dabei dient der Bluetooth-Empfänger als Bus-Master und generiert somit die Taktsignale. Wie in Abb.4.1 zu sehen, empfängt der Esp32s3 das Datensignal des Empfängers und schreibt die beiden Datensignale für die DACs.

4.3 Programmstruktur

Die verwendete Anwendung wurde in der Programmiersprache C entwickelt. Dafür wurde die IDE Visual Studio Code mit der ESP-IDF-Erweiterung verwendet. Bei den verwendeten Software *librarys* handelt es sich entweder um Standard *librarys* oder um *librarys* entwickelt von Espressivo Systems, die zur Konfiguration und Steuerung der verwendeten Hardware benutzt werden. Für die Implementierung der digitalen Filter wurden keine *librarys* verwendet.

Der Mikrocontroller erfüllt drei Aufgaben, das Empfangen und Senden von Datensignalen über den I2S-Bus, die Filterung des Eingangssignals und das Einlesen der Potenziometers zur Steuerung der Lautstärke. Da der DSP bei 48 kHz eingesetzt werden soll, muss die Filterung sowie das Empfangen und Senden einzelner Werte auch bei dieser Frequenz stattfinden. Somit muss dieser Prozess in 20,8 Mikrosekunden stattfinden. Bei einem Prozessortakt von 240 MHz bedeutet das, dass 5000 CPU-Zyklen für diesen Prozess zur Verfügung stehen. Damit der Programmcode, der für diese Aufgaben verwendet wird, in dieser Zeit ausgeführt werden kann, muss dieser mit einigen Einschränkungen entwickelt werden. Beispielsweise werden hier ausschließlich Integer-Datentypen verwendet und *floating-point* Operationen werden vollständig vermieden.

Das Einlesen der Potenziometer zur Steuerung der Lautstärke muss jedoch nicht bei einer so hohen Frequenz stattfinden. Auch können hier *floating-point* Operationen kaum vermieden werden. Das sich aus den unterschiedlichen Anforderungen ergebene Problem wird umgangen, indem die Anwendung auf einem *dual-core* Prozessor implementiert wird. Dabei ist ein Kern dafür zuständig, die Filterung durchzuführen und die I2S-Schnittstelle zu verwenden, während der andere Kern die Potentiometer einliest und diese Signale verarbeitet.

4.4 Implementierung digitaler Filter

Die Umsetzung der digitalen Filter besteht vorwiegend aus der Implementierung von Gl.3.3. Die Filterparameter a_n und b_n wurden mithilfe von MATLAB berechnet. Ein Butterworth-Filter zweiter Ordnung wird durch fünf Koeffizienten definiert. Die zwei Koeffizienten des Nennerpolynoms a_n werden mit den entsprechenden Werten der letzten Ergebnisse y_n multipliziert, während die drei Koeffizienten des Zählerpolynoms b_n mit dem aktuellen und mit vorherigen Eingangssignalen x_n multipliziert werden. Das Ergebnis des Filters besteht dann aus der Summe dieser Produkte. Jeder der vier Signalkanäle wird mit einem Bandpassfilter zweiter Ordnung

gefiltert. Dafür durchlaufen die Signale zuerst einen Hochpassfilter und anschließend einen Tiefpassfilter. Insgesamt müssen in jedem Schritt acht Filter berechnet werden. Um zu gewährleisten, dass die Berechnung aller Filter bei den benötigten 48 kHz ausgeführt werden kann, wurden die Filter mit Integer-Arithmetik implementiert.

Rechnungen mit Integer-Datentypen beanspruchen deutlich weniger Zeit als Rechnungen mit Gleitkommazahlen. Letztere sollten also vermieden werden, wenn Performance von hoher Wichtigkeit ist. Da die Koeffizienten des Filters aber Gleitkommazahlen sind, werden diese mit einem Faktor k multipliziert und auf den nächsten Integer gerundet. Nachdem die Multiplikation stattfindet, wird das Ergebnis wieder durch k dividiert. Da durch die Rundung ein Fehler entsteht, sollte der Faktor k so groß wie möglich gewählt werden. Um die Division so performant wie möglich implementieren zu können, wird für den Faktor k oft eine Zweierpotenz gewählt. Dadurch kann die Division mithilfe des *bit-shift right* Operators „ $\gg$ “ durchgeführt werden. Diese Operation „schneidet“ die spezifizierte Menge an Bits n an der linken Seite einer Zahl ab. Es erfolgt also eine Division der Zahl durch 2^n . Das Eingangssignal ist ein 32-Bit-Integer. Um einen *overflow* durch die Multiplikation zu vermeiden, sind die Arrays der Ein- und Ausgangssignale 64-Bit-Integer. In dieser Implementierung wurde $k = 2^{30}$ gewählt.

Eine simplifizierte Implementierung des Filters ist in List. 4.1 zu sehen. Zuerst werden die Arrays der letzten Eingangs- und Ausgangswerte um eins verschoben. Danach wird das neue Eingangssignal an erste Stelle in das Eingangsarray geschrieben. Anschließend erfolgt in Zeile 19-23 die eigentliche Berechnung des Filters.

List. 4.1: Simplifizierte Filterimplementierung

```

1 //-----Mid Coeffs-----//
2 const int32_t MidCoeffs_A_HP[2] = { 2106887749, -1033695662 }; //hp203
3 const int32_t MidCoeffs_B_HP[3] = { 1053623222, -2107246444,
   1053623222 };
4
5 int64_t lastX[3] = {0};
6 int64_t lastY[3] = {0};
7 const int shift = 30;
8
9 void calculateY(int32_t newX) {
10     for (size_t i = 2; i >= 1; i--) {
11         lastY[i] = lastY[i - 1];
12         lastX[i] = lastX[i - 1];
13     }
14
15     lastX[0] = newX
16
17     //--HP_203Hz--//
18     lastY[0] = ((lastX[0] * MidCoeffs_B_HP[0]) >> shift) +
19     ((lastX[1] * MidCoeffs_B_HP[1]) >> shift) +
20     ((lastX[2] * MidCoeffs_B_HP[2]) >> shift) +
21     ((lastY[1] * MidCoeffs_A_HP[0]) >> shift) +

```

```

22 ((lastY[2] * MidCoeffs_A_HP[1]) >> shift);
23 }

```

Das Programm berechnet in einem Schritt zwei Eingangswerte und berechnet jeweils zwei neue Ausgangswerte für jeden der vier Audiokanäle. Dieser Schritt kann nun in etwa 1325 Prozessorakten durchgeführt werden, was etwa 5,52 Mikrosekunden entspricht und weit unter der maximalen Zeit von 20,8 Mikrosekunden pro Eingangswert liegt. Auch die Anwendung der Gesamtlautstärke und die der einzelnen Frequenzbereiche wird mit Integer-Arithmetik umgesetzt.

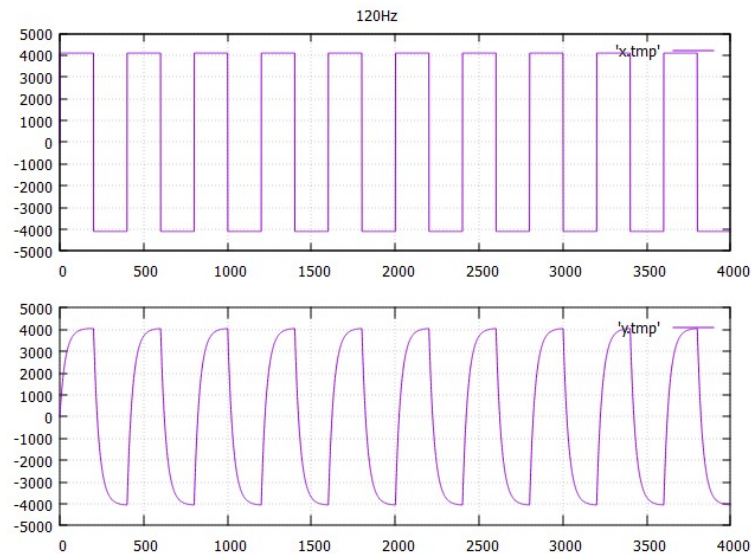


Abb. 4.2: Ausgabe des Testprogramms. Oben das Eingangssignal unten das gefilterte Ausgangssignal. Ein Rechtecksignal wird mit einem Tiefpassfilter gefiltert

Um die Entwicklung der Filter zu vereinfachen, wurde ein Programm geschrieben, das Filter auf ein simuliertes Eingangssignal anwendet und das Ergebnis visualisiert. Das Programm wurde nicht für den Mikrocontroller entwickelt, sondern läuft auf einem Windows-Computer. Da es etwa eine Minute braucht, um ein Programm für den Esp32 zu kompilieren und diesen zu *flashen*, kann sich dadurch viel Zeit bei der Entwicklung gespart werden. Des Weiteren sind die *debugging* Möglichkeiten auf dem Esp32 stark eingeschränkt. Ohne einen JTAG-Adapter zu verwenden, ist *debugging* nur durch Textausgabe über die serielle Schnittstelle möglich. Das Testprogramm wurde verwendet, um die Stabilität und Funktionsweise verschiedener Filter schnell testen zu können. Dadurch wird die Möglichkeit von Fehlern durch Wackelkontakte, I2S Fehler oder sonstige potenzielle Probleme ausgeschlossen. Das Programm wurde auch in C entwickelt und die Visualisierung wurde mit gnuplot umgesetzt (Abb.4.2).

4.5 Lautstärkesteuerung

Die Potentiometer werden mit einer Frequenz von 100 Hz vom Mikrocontroller eingelesen. Um zu verhindern, dass sich Rauschen auf den Potentiometern auf das Audiosignal auswirkt, werden auch diese Signale gefiltert. Dafür werden digitale Butterworth-Tiefpassfilter der zweiten Ordnung mit einer Grenzfrequenz von 1 Hz verwendet. Bei der vorherigen Implementierung war ein niederfrequentes „Brummen“ zu hören, wenn die Drehknöpfe berührt wurden. Durch die Filterung soll dieses Problem behoben werden.

Um die Lautstärke um 3 dB zu erhöhen, ist eine Verdopplung der Leistung nötig. Deshalb werden in Audiosystemen oft Potentiometer mit einer logarithmischen Kennkurve eingesetzt. Dadurch sorgt eine Änderung der Position des Potentiometers um einen gewissen Winkel immer für die anteilmäßig gleiche Änderung der Lautstärke, unabhängig von der ursprünglichen Stellung des Potentiometers. Würde man einen linearen Potenziometer verwenden, würde sich die Spannung und somit die Leistung immer anteilmäßig gleich verändern, nicht aber die Lautstärke.

Um das Verhalten der Potentiometer durch Software definieren zu können, wurden lineare Potentiometer verbaut. Um ein logarithmisches Verhalten des Potentiometers für die Gesamtlautstärke zu verursachen, wurde eine Formel auf das Signal des Potentiometers angewandt. Der Esp32s3 verfügt über 12-Bit ADCs der maximale Wert beträgt somit $2^{12} = 4096$. Die Lautstärke l wird anhand des Signals x gewonnen:

$$l = x \left(\frac{x}{4096} \right)^2 \quad (4.1)$$

Um bei den Potentiometern für die einzelnen Frequenzbereiche mehr Feingefühl zu gewährleisten, wurden die Signale dieser Potentiometer ohne weitere Verarbeitung verwendet. Dadurch verursacht eine Änderung des Potentiometers im hohen Bereich eine deutlich kleinere Änderung der Lautstärke als eine gleich große Änderung im niedrigen Bereich. Das ist sinnvoll, weil meist nur kleine Änderungen an den Pegeln der Frequenzbereiche nötig sind. Eine große Änderung einzelner Pegel führt nicht zu guter Klangqualität, kann aber interessant sein, um den Effekt der einzelnen Frequenzbereiche zu verdeutlichen. Der Einsatz von linearen Potentiometern erlaubt Feingefühl bei hohem Pegel, es ist aber auch möglich, einzelne Frequenzbereiche ganz auszuschalten.

4.6 Validierung

Um die Funktionsweise der Filter und des restlichen Systems zu validieren, wurden Messungen des Amplitudengangs des Systems durchgeführt. Meistens werden Messungen von Audiosystemen in sogenannten „schalltoten“ Räumen durchgeführt, wobei spezialisierte und teure Hard- und Software eingesetzt wird. Da solche Ausrüstung nicht zur Verfügung stand, wurden die

Messungen mit einem Smartphone in einem Raum ohne Schalldämmung aufgenommen. Dabei wurde die kostenlose App Spectroid verwendet. Aufgrund der mangelhaften Qualität der Messausrüstung und der Störeinflüsse durch Umgebungsgeräusche und Resonanzen dienen die erhaltenen Ergebnisse lediglich zur groben Einordnung des Systemverhaltens. Durch den Vergleich verschiedener Messungen kann das Verhalten verschiedener Filter verglichen werden, genaue oder absolute Aussagen können aber nicht getroffen werden.

Alternativ könnte zur Validierung auch die Ausgangsspannung der DACs gemessen werden. Dadurch würden die meisten Fehlerursachen eliminiert werden und es könnte eine stärkere Aussage bezüglich der Funktionsweise der Filter getroffen werden. Das Verhalten des gesamten Systems kann durch so eine Messung jedoch nicht untersucht werden.

5 Ergebnisse

Die Ergebnisse der in Kap.4.6 beschriebenen Messungen sind in Abb.5.1 abgebildet. Trotz der qualitativ mangelhaften Messumstände ist die Funktionsweise der Filter erkennbar. Der Hochpassfilter des Subwoofers schützt diesen vor Beschädigung. Die Grenzfrequenz von 17 Hz kann von diesem nicht erreicht werden und der Abfall der Amplitude bei 50 Hz entspricht der Spezifikation des Herstellers. Der Tiefpassfilter der Hochtöner liegt bei 23 kHz und ist somit außerhalb des Messbereichs der verwendeten App. Im Amplitudengang der Mitteltöner sind im Durchlassbereich einige Einbrüche des Pegels zu erkennen. Gründe dafür könnten Resonanzen des Messraums, unerwünschte Resonanzen im Gehäuse durch mangelhafte Konstruktion oder schlechte Qualität der Lautsprecher selbst sein. Da für die Mitteltöner vergleichsweise kostengünstige Lautsprecher gewählt wurden, ist letzterer Punkt durchaus eine Möglichkeit. Da sich dieser Effekt auch im Amplitudengang des ganzen Systems ausdrückt, sollte dem in zukünftigen Weiterentwicklungen nachgegangen werden. Aufgrund der restlichen Ergebnisse ist es jedoch unwahrscheinlich, dass die Einbrüche durch den DSP erzeugt werden. Auch die Ergebnisse der Simulation mit dem Testprogramm unterstützen diese Vermutung.

Durch den Einsatz des DSP konnten die Probleme des bisherigen Systems beseitigt werden. Die Komplexität des Signalpfads ist auf das Mindeste reduziert, und die Verstärkungen der einzelnen Frequenzbereiche können weiterhin unabhängig eingestellt werden. Der analoge Signalpfad besteht inzwischen nur aus einem Kabel vom Ausgang der DACs zu den Eingängen der Audioverstärker. Dadurch können verschiedene Störeinflüsse vermieden werden. Im Vergleich zum vorherigen System werden deutlich weniger Kabel im Gehäuse benötigt. Dadurch werden Fehlerursachen reduziert und der Zugang zu einigen Komponenten wird erleichtert. Über eine Mikro-USB-Buchse an der Oberseite des Gehäuses hat man Zugang zum Esp32s3 und kann die Filter oder andere Teile des Programms jederzeit und mit geringem Aufwand ändern, ohne dass das Gehäuse geöffnet werden muss.

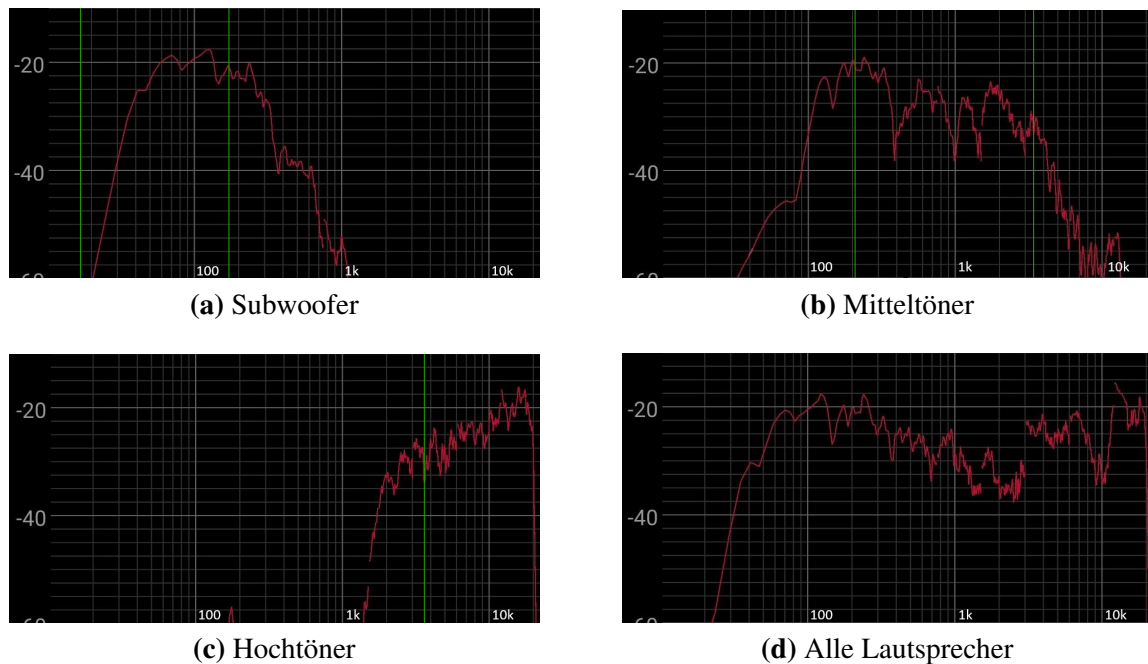


Abb. 5.1: Messung verschiedener Amplitudengänge. In (a), (b) und (c) ist jeweils nur ein Lautsprecher aktiv, während in (d) alle eingeschaltet sind. Die Grenzfrequenzen der Filter sind in grün eingezeichnet.

Aufgrund der hohen benötigten Performance der Filterimplementierungen musste das verwendete Programm stark optimiert werden. Das wurde vor allem durch die Verwendung von Integer-Arithmetik erreicht. Dadurch konnte der DSP umgesetzt werden, ohne dass Qualitätseinbußen durch Reduzierung der Auflösung oder Frequenz der Filterung in Kauf genommen werden mussten. Durch diese Maßnahmen wurde die Klangqualität spürbar verbessert. Auch die maximale Lautstärke wurde deutlich erhöht und ist nun für die allermeisten Anwendungsfälle ausreichend. Die Taster, um das aktuelle Lied pausieren, überspringen und wiederholen zu können (Abb.2.1), konnten durch das neue System umgesetzt werden und verbessern die Nutzererfahrung noch weiter.

Ein Nachteil des neuen Systems ist, dass die Hochtöner nicht mehr in *stereo* eingesetzt werden können. Dieser Nachteil wirkt sich aber kaum negativ auf das Hörerlebnis aus. Die Hochtöner sind so nah aneinander montiert, dass der *stereo* Effekt auch zuvor nur in nächster Nähe zur Musikbox zu erkennen war.

6 Zusammenfassung und Ausblick

Das Projekt hat die anfangs gestellten Ziele erfüllen können, und die Musikbox wurde durch zusätzliche Funktionen erweitert. Insgesamt hat das Projekt gezeigt, dass der Einsatz eines DSP in einem Audiosystem erhebliche Vorteile bieten kann. Die gewonnenen Erkenntnisse und Erfahrungen können als Grundlage für weitere Projekte und Optimierungen dienen. Die Steigerung der Klangqualität lässt sich schlecht quantitativ beschreiben, ist aber nicht zu übersehen.

Da durch die effiziente Implementierung der Filter noch Rechenleistung „übrig“ ist, könnte diese in einer Weiterentwicklung verwendet werden, um komplexere Filter zu implementieren. Beispielsweise könnten Filter einer höheren Ordnung eingesetzt werden.

Der Esp32s3 verfügt über eine Antenne, die Bluetooth oder WLAN unterstützt. Diese bleibt in diesem Projekt ungenutzt. Nach der Entwicklung einer entsprechenden Anwendung für ein Smartphone könnte die Antenne verwendet werden, um sich mit diesem zu verbinden. Die Anwendung könnte genutzt werden, um die Werte der verschiedenen Lautstärken zu steuern oder sogar um die Parameter der Filter während des Betriebs zu ändern. Da viele Eigenschaften der Musikbox durch Software definiert werden, bieten sich hier viele Möglichkeiten.

Nach der Fertigstellung des Projekts wurde dieses über einen Zeitraum von etwa sechs Monaten ausgiebig unter verschiedenen Umständen getestet und erprobt. Dabei konnten keine Mängel am Signalverarbeitungssystem festgestellt werden. Es wurden jedoch kleinere Mängel am Gehäuse sowie an der Stromversorgung der Audioverstärker identifiziert. Diese Mängel werden in zukünftigen Weiterentwicklungen adressiert.

Literaturverzeichnis

- [1] Stephen Butterworth et al. On the theory of filter amplifiers. *Wireless Engineer*, 7(6):536–541, 1930.
- [2] Connor W Herron. Software implementation of digital filtering via tustin’s bilinear transform. *arXiv preprint arXiv:2401.03071*, 2024.
- [3] NXP Semiconductors. *I²S Bus Specification*, rev. 3.0 edition, February 2022.

Abbildungsverzeichnis

2.1	Die Musikbox in dem der DSP eingesetzt wird	2
3.1	I2S Zeitverlaufsdiagramm [3]	5
4.1	Hardware Aufbau der relevanten Komponenten. Links der Bluetooth-Empfänger, mittig der Esp32s3 und rechts die zwei DACs	7
4.2	Ausgabe des Testprogramms. Oben das Eingangssignal unten das gefilterte Ausgangssignal. Ein Rechtecksignal wird mit einem Tiefpassfilter gefiltert	10
5.1	Messung verschiedener Amplitudengänge. In (a) , (b) und (c) ist jeweils nur ein Lautsprecher aktiv, während in (d) alle eingeschaltet sind. Die Grenzfrequenzen der Filter sind in grün eingezeichnet.	13

Listings

4.1	Simplifizierte Filterimplementierung	9
-----	--	---